
Predicting Outcomes in Professional League of Legends Matches

Joshua Hsieh ^{* 1} Joseph Bailey ^{* 2}

Abstract

In this paper we develop a model to predict the outcomes of professional League of Legends matches. We use several different models and input vector formats to optimize our success rate. While doing so, we identify several significant statistics that have previously been unused, and are able to achieve a higher classification accuracy than prior work has achieved.

1. Introduction

League of Legends is the largest esport in the world, with over 150 million registered players and 10 million active players each day. The professional scene is similarly popular, with matches drawing up to 8 million concurrent viewers. Professional League of Legends has been around for more than a decade, and thousands of matches per year offer a plethora of data that can be utilized. As with any sport, a prominent problem is determining which team will win in any given match. Our project aims to apply machine learning to determine how a team can maximize their odds of winning. In League of Legends, the two competing teams participate in a draft, choosing the champions that they will play. Champions are the characters that players control, and each one has unique abilities that serve different purposes. Due to the enormous amount of possible team compositions and interactions between champions, having a successful draft is a crucial part of League of Legends. Our goal is to predict which team has the higher chance of winning, given which champions they have drafted.

2. Related Work

Plenty of academic work has been done on win prediction in League of Legends; however, very few study pre-match data in professional games.

^{*}Equal contribution ¹Department of Computer Science, Stanford University ²Department of Physics, Stanford University. Correspondence to: Joseph Bailey <josephrb@stanford.edu>, Joshua Hsieh <joshhsieh@stanford.edu>.

Do et al. found that a deep neural network can predict League of Legends match outcomes with 75.1% accuracy (2021). This work is similar to ours in that it only uses data from before the match has started. However, this study didn't consider professional matches, only investigating ranked games, which can be played by anyone. As inputs, they used champion mastery points and player-champion win rate. They found that the season total number of games played on the picked champion and number of recent games played on the picked champion negatively impacted model performance. They also added additional meta-features: the "team's average, median, coefficient of excess kurtosis (i.e., how normal the distribution is), coefficient of skewness, standard deviation, and variance of the players' champion win rate and of their champion mastery points," (Do et al., 2021) for a total of 44 features per match. Out of SVC, KNN, RF, GBOOST, and DNN, they found DNN with dropout layers to be the most suitable.

Ani et al. is most similar to our work, analyzing 1500 professional League of Legends matches using the pre-match features: champion, bans, summoner spells, role (2019). They used the within-match features of first blood, gold spent percentage difference, KD (kill/death ratio), CSat10 (creep score at 10 minutes), CSdat10 (creep score difference at 10 minutes), Expat10 (experience earned at 10-minutes), KPM (kills per minute). Similarly to Do et al., they also used feature selection algorithms. More specifically, they used recursive feature elimination with cross validation, ranking top-3 features by Gini importance. Out of RF, Adaboost, GB, and XGBoost, RF had the best prematch accuracy of 95.52%, mainly due to the "ban" feature. Using both pre- and within-match data, they achieved a combined accuracy of 99.75% with RF.

Similarly, Costa et al. analyzed the pre-match data of professional League of Legends matches, using 51 features containing information about champion bans, champions picked, and stats of each player with respect to their picked champion (e.g. win rate, games played, ratio of kills + assists per death) (2021). They used 10-fold cross-validation with each algorithm and found RF, SVM, and LR to outperform DT, NB, and kNN. Contradicting Ani et al., they concluded that the bans and the specific champion picks are unimportant on their own, rather finding player statistics the most impactful with an AUC of 0.9. Out of all player

statistics, they found their win rate percentages and KDA on their played champion to be the most influential. They had a total accuracy of 0.91 and AUC of 0.97, which we were able to improve upon.

3. Dataset and Features

3.1. Dataset

Our data was retrieved from Oracle’s Elixir ([Sevenhuysen](#)), a League of Legends analytics website that has collected data from professional matches since 2014. Each annual season comes with a major update to the game, changing the stats on each champion with the aim of rebalancing. Thus, we chose to conduct our analysis on data purely from 2024.

For each match, Oracle’s Elixir contains data on the two teams playing in it, along with match data collected throughout the match. More specifically, what’s recorded are the five players on both teams, the five champions they are playing, and the five champions they banned. There is also in-game data, which encapsulates each team’s performance throughout the game. However, since our aim is to predict the winning team only with information found from before the match begins, we ignore information collected after players’ picks and bans have been chosen.

The dataset contains data for 8562 matches, which represent almost all professional League of Legends matches played in the 2024 season. However, our model’s focus is predicting the outcomes of matches in major leagues, mainly the Korean, Chinese, European, and North American leagues. Thus, because many of these leagues have their own trends and strategies that differ from other leagues, data involving teams in less relevant leagues is less helpful to our model. Furthermore, we had to prune some matches due to incomplete data.

In total, we have 2652 unique players and 528 unique teams. The distribution of the number of occurrences of these players and teams can be found in Figure 1. Both distributions are right-skewed, which is reflective of how the dataset contains many teams and players from minor leagues, which play far less games than the major leagues.

Our dataset also saw 168 distinct champions played. The distribution of the number of occurrences of these champs can be found in Figure 2. This distribution is also right-skewed, which is reflective of how there are a few very popular “meta” champions, which are played at very high frequencies, while other champions are much less popular. Figure 3 shows the distribution of season win rates for each player. This seems to be normally distributed, which lends credibility to our use of it in our models.

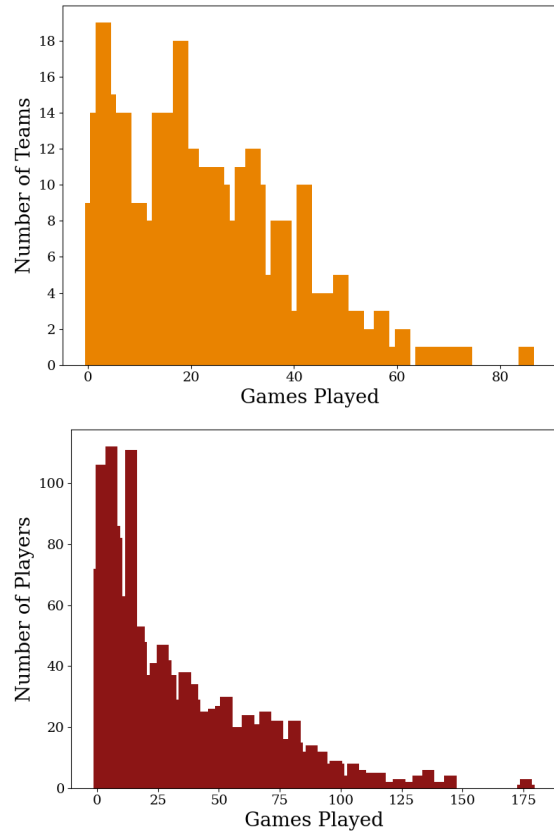


Figure 1. Games played by each player and team, 2023-24 season.

3.2. Features

From the features provided in our dataset, we experimented with several different formats of input vectors. Below are descriptions of each of our attempts.

3.2.1. ONE-HOT ENCODING

We formatted inputs as one-hot vectors that encode team composition, with an input vector for each team before a match. This vector contains 169 dimensions, with one dimension for each possible champion. For each dimension, we put 1 if team is playing the corresponding champion and 0 if not. The corresponding output feature is 1 or 0, representing if the team won or lost.

3.2.2. ONE-HOT ENCODING WITH WINRATES

Building upon the last format of input vector, we created an input vector for each team before a match. As before, this vector also has 169 dimensions, one dimension for every possible champion. For each dimension, we put the corresponding win rate on that champion of the player who picked that champion, or 0 if the champion is not being

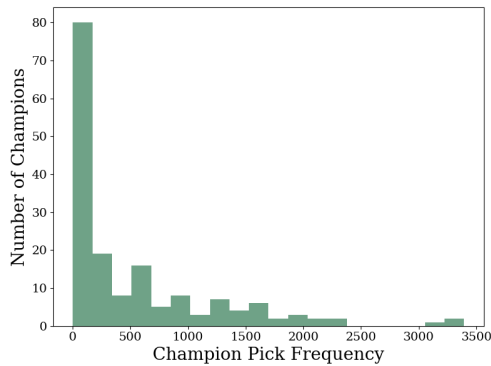


Figure 2. Number of times each champion was picked, 2024 season.

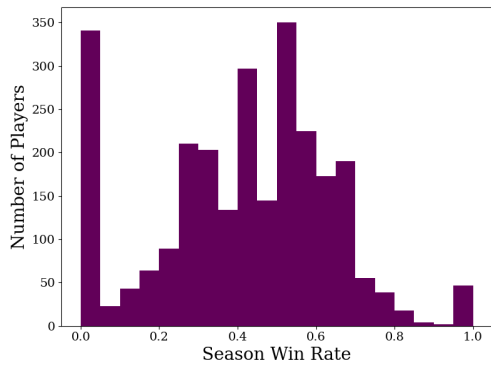


Figure 3. The winrate of each player, 2024 season.

played. For example, suppose the first dimension corresponds to Ezreal, a champion. Then, if player X is playing Ezreal this match and has a career 70% win rate with him, the first entry of the vector will be 0.7. The corresponding output feature is 1 or 0, representing if the team won or lost.

3.2.3. TEAM VECTORS

Rather than having a dimension for every possible champion, many of which are 0, we to reduce the dimensions and also remove champion specific dimensions from the input vectors. Instead, each team before a match is represented by a 20 dimension matrix. Each of the 5 players on the team have 4 corresponding entries in the input vector, which store the player’s winrate on the champion they are playing, the player’s winrate over all games played, their winrate of their played champion over all games that it has been played in, and the winrate of their banned champion over all games that it has been played in. The corresponding output feature is 1 or 0, representing if the team won or lost.

3.2.4. MATCH VECTORS

In order to incorporate information about both teams playing in a match, we combined the team vectors of the two teams facing each other into one vector representing the entire match. This resulted in a 40 dimension vector, with 4 dimensions for each of the 10 players in the match. In each match one team is on the “Blue” side and the other team is on the “Red” side. We placed the blue side team first and the red side team second in the match vector. The corresponding output feature is 1 or 0, representing if the blue side team won or lost.

3.2.5. MATCH VECTORS WITH EMBEDDINGS

By choosing to fill our input vectors with winrates, we lose nuance with respect to the team composition. That is, our input data no longer contains any information about the actual champions picked. Since we didn’t want to add another 169 dimensions to our input vectors, we decided to creating embeddings for each team composition. To do so, we trained a Word2Vec model, treating each team composition as an independent sequence based on pick order. Using a context window size of five — the size of each team, we created eight-dimensional embeddings. These embeddings were appended to the match vectors from the previous subsection.

4. Methods

We tried a variety of models, all of which we trained using 80% of the dataset, and tested with the remaining 20% of the dataset. We implemented these models with the scikit-learn library.

4.1. Logistic Regression

We used logistic regression, which assumes a linear relationship between the features and outputs by learning a linear decision boundary in the data. In combination with our basic one-hot encoding input vector frame, this forms our baseline model. The success of this model depends on if the data is inherently linearly separable.

4.2. Gradient Boosting

Gradient boosting is an ensemble model where worse models are iteratively weighted using training data to create a larger, more accurate model. These weights are updated using the residuals of the last model. The ensembling of many weak learners results in a strong learner. Our model used 100 estimators, a learning rate of 0.1, and a max depth of 3.

4.3. Random Forest

Random forests is an ensemble model that combines several decision trees. Decision trees make predictions by making choices based on data features, splitting the data until reaching a single class, which is the prediction. Random forests are similar to bagging, but decrease correlation between individual decision subtrees by only considering a random subset of features at each splitting step in the algorithm. Our model used 100 estimators.

4.4. Neural Network

Neural Networks consist of layers of fully-connected neurons, each of which has a non-linear activation function applied to it. There are several possible types of activation functions, of which we used the ReLU function for the hidden layers and the sigmoid function for the final output layer. Our model had hidden layers with 128, 64, 32 neurons. We used the Adam optimizer, L2 regularization, batch normalization, and dropout on each hidden layer to optimize our model. We also used a learning rate scheduler to decrease the learning rate after 5 epochs. Our model had a batch size of 32 and ran for 50 epochs.

4.5. Experiments

To fully test our input vector formats and methods, we ran all input vector formats against all methods. To ensure consistency, we used the same train/test split when using the same input vector formats.

5. Results and Discussion

5.1. Evaluation Metrics

For our classification model, we consider the number of True Positives, True Negatives, False Positives, and False Negative resultss. True Positives are results that are correctly predicted as 1. True Negatives are results that are correctly predicted as 0. False Positives are results that are incorrectly predicted as 1. False Negatives are results that are incorrectly predicted as 0.

5.1.1. CLASSIFICATION ACCURACY

Classification accuracy measures the overall accuracy of a classification model on a dataset. This is equal to

$$\frac{\text{True Positives} + \text{True Negatives}}{\text{Number of Predictions}}$$

5.1.2. AUC

AUC is another measure of the accuracy of a classification model. It measures the area under the ROC (reciever operating characteristics) curve, which plots the rate of True

Postives against the rate of False Positives. This measures how well a model is able to distinguish between classes.

5.1.3. PRECISION

Precision represents the percent of predicted positive cases which are true. The formula for this metric is

$$\frac{\text{True Postives}}{\text{True Positives} + \text{False Positives}}$$

5.1.4. RECALL

Recall represents the percent of real positive cases which are predicted. The formula for this metric is

$$\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

5.1.5. F1 SCORE

F1 score represents a combination of Precision and Recall. The formula for this metric is

$$\frac{2 \cdot (\text{Precision} + \text{Recall})}{\text{Precision} + \text{Recall}}$$

5.2. Results

Tables 1, 2, and 3 summarize the evaluation metrics for our four models and five input vector formats. All of our models were able to achieve at least 95% accuracy, AUC, and F1 score. Adding embeddings to the match vectors tended to not significantly improve any metrics. However, RF did achieve the top accuracy of 96% using the match vectors and embeddings as input. NN also consistently outperformed all the other models on AUC, ultimately reaching 99% AUC.

Figure 4 consists of plots of the accuracy and loss when running our NN on match vectors plus embeddings. These plots successfully demonstrate convergence, with the validation accuracy and loss outperforming the training set. This is due to our use of dropout and regularization in the hidden layers.

Figure 5 graphs precision-recall for all models with match vectors and embeddings as input. These graphs are very exceptional, demonstrating the high performance of all models.

5.3. Discussion

Most of our models had high accuracy, but the models that used the Match vector input format were the most successful. This seems to be due to our usage of the champion specific player win rate, which we believe to be the most significant feature in predicting success. Other features such as season player win rate and season champion win rate,

Table 1. Classification Accuracy.

INPUT	LR	GB	RF	NN
BASILINE (ONE-HOT)	0.53	0.52	0.51	0.54
ONE-HOT WINRATE	0.82	0.75	0.78	0.82
TEAM VECTORS	0.91	0.91	0.91	0.91
MATCH VECTORS	0.95	0.95	0.95	0.95
MATCH VECTORS + EMB	0.95	0.95	0.96	0.95

Table 2. AUC.

INPUT	LR	GB	RF	NN
BASILINE (ONE-HOT)	0.53	0.52	0.51	0.55
ONE-HOT WINRATE	0.82	0.76	0.78	0.92
TEAM VECTORS	0.91	0.91	0.91	0.98
MATCH VECTORS	0.95	0.95	0.95	0.99
MATCH VECTORS + EMB	0.95	0.95	0.95	0.99

Table 3. F1 Score.

INPUT	LR	GB	RF	NN
BASILINE (ONE-HOT)	0.51	0.47	0.49	0.51
ONE-HOT WINRATE	0.82	0.73	0.77	0.82
TEAM VECTORS	0.91	0.91	0.91	0.91
MATCH VECTORS	0.95	0.95	0.95	0.95
MATCH VECTORS + EMB	0.95	0.95	0.95	0.95

which we also use in bans, are also important. Even though our initial thought was that it would be important to consider synergies between pairs of champions, it seems that directly adding features for each champion is not useful. Rather, we believe champion and player win rates encapsulate these synergies for us, since professional teams draft champions with compatability in mind. As a result, win rate is a simple yet powerful way to discern overall team strength. Another possibility is that because professional teams rarely play with weak team compositions, considering inter-champion relationships may not be useful.

6. Conclusion/Future Work

All of our models achieved high accuracy, in all three of the metrics we used, which improves upon existing models in the literature.

For future work, we would like to find some way to utilize the datasets from previous years, which could increase the accuracy of the model, despite possibly being outdated. Furthermore, we believe that cleaning the data, by removing data corresponding to less relevant teams and players, could also improve our model.

7. Contributions

Joshua worked on compiling statistics from the dataset, testing various models with different input formats, and vi-

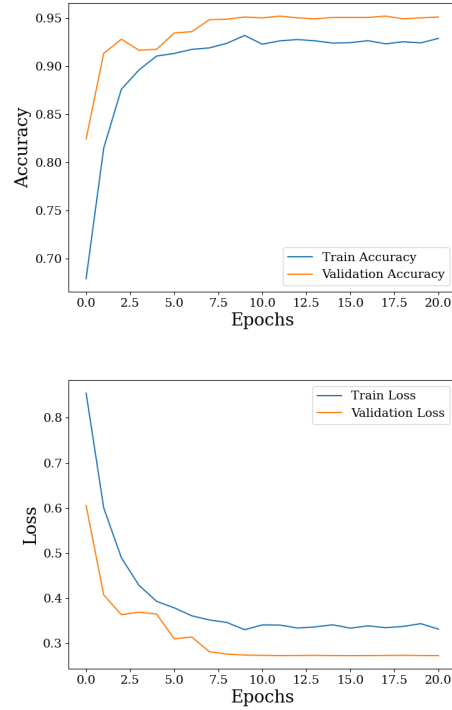


Figure 4. Train/validation accuracy and loss for the NN.

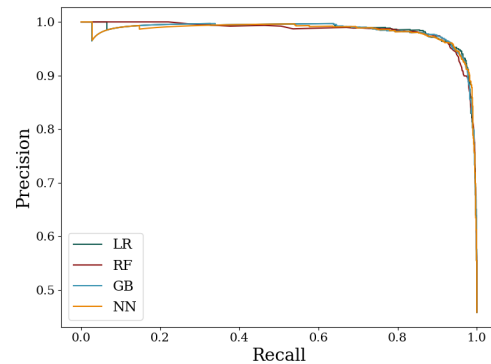


Figure 5. Precision-recall using Match Vectors + EMB.

sualization of results. Joseph also tested various models, including neural networks, added champion embeddings in the input vectors, and visualization of results.

References

- Ani, R., Harikumar, V., Devan, A. K., and Deepa, O. Victory prediction in league of legends using feature selection and ensemble methods. In *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*, pp. 74–77, 2019. doi: 10.1109/ICCS45141.2019.9065758.
- Costa, L., Mantovani, R., Souza, F. C., and Xexéo, G. Feature analysis to league of legends victory prediction on the picks and bans phase. In *2021 IEEE Conference on Games (CoG)*, 08 2021. doi: 10.1109/CoG52621.2021.9619019.
- Do, T. D., Wang, S. I., Yu, D. S., McMillian, M. G., and McMahan, R. P. Using machine learning to predict game outcomes based on player-champion experience in league of legends. In *The 16th International Conference on the Foundations of Digital Games (FDG) 2021, FDG'21*, pp. 1–5. ACM, August 2021. doi: 10.1145/3472538.3472579. URL <http://dx.doi.org/10.1145/3472538.3472579>.
- Sevenhuysen, T. Oracle's elixir - lol esports stats. from <https://oracleselixir.com/>.